Classical Optimization Algorithms for Lasso Regression

A Review and Empirical Comparison

COMP6704 Group Project — Group 9

Convex Optimization | 7 October 2026

CAI Yangxiao*
The Hong Kong Polytechnic University
Student ID: 26023041R
26023041r@connect.polyu.hk

XUE Pengyu*
The Hong Kong Polytechnic University
Student ID: 26022951R
26022951r@connect.polyu.hk

ZHANG Kejia*
The Hong Kong Polytechnic University
Student ID: 26049131R
26049131r@connect.polyu.hk

ZHOU Xiyu*
The Hong Kong Polytechnic University
Student ID: 26025265R
26025265r@connect.polyu.hk

Abstract

Lasso regression combines squared loss with an ℓ_1 penalty to learn sparse linear models. We organize classical solvers by vector updates, coordinate minimization, path construction, and splitting or curvature-based steps. Basic analysis connects convexity, subgradient optimality, and soft thresholding; related penalties clarify the difference between numerical convergence and statistical selection. The empirical comparison implements ISTA, standard FISTA, and cyclic coordinate descent. Experiments on three synthetic instances and one fixed Diabetes split compare implementations under a common stationarity tolerance. All methods converge to nearly identical objectives and predictions. ISTA is fastest on the original synthetic instances; coordinate descent is fastest on Diabetes. Controlled correlation and dimension studies test whether this ordering persists. FISTA has no time advantage on the original benchmark but is faster on average at within-block correlation 0.99. Coefficient plots reveal shrinkage despite exact support recovery in the synthetic cases, while a penalty study illustrates the fit–sparsity trade-off. Results are limited to the recorded datasets, implementations, and stopping rule.

1 Introduction

When a regression model has many candidate predictors, a small training error can come with coefficients that are difficult to interpret. Lasso adds an ℓ_1 penalty to the squared loss, shrinking coefficients and setting some exactly to zero (Tibshirani, 1996). This convex linear model offers a simple setting for examining both optimization and feature selection.

The resulting objective is convex but nonsmooth at zero. Convexity excludes suboptimal local minima; it does not determine how quickly a solver reaches a useful accuracy. The comparison therefore turns on how an algorithm handles the penalty and uses the structure of the least-squares loss.

We compare ISTA, standard FISTA, and cyclic coordinate descent at the same penalty and stationarity tolerance to explain differences in convergence and computation time. A separate experiment varies the penalty to examine test error and selected variables. Keeping these comparisons separate distinguishes the effect of regularization from the cost of solving a fixed training problem.

*All authors contributed equally and are listed in alphabetical order by surname.

1.1 Scope and literature selection

This focused narrative review concerns convex squared-loss Lasso for linear regression. It groups methods by the information used in an update and the work required to reach an accuracy target. Foundational model and solver papers are supplemented by related penalties, restarting, and safe screening. Experiments retain the three basic solvers for the same squared-loss objective.

The literature connects the three updates through a common structure (Table 1). Tibshirani (1996) introduced Lasso's shrinkage and selection formulation; Daubechies et al. (2004) studied iterative thresholding for inverse problems. Beck and Teboulle (2009) established the acceleration comparison, while Friedman et al. (2010) developed coordinate updates for regularized regression. Parikh and Boyd (2014) explains the proximal framework and Wright (2015) reviews coordinate selection rules and their assumptions. LARS supplies context on solution paths and Diabetes data (Efron et al., 2004); computing an entire path is outside our fixed-penalty benchmark.

The review also separates numerical convergence from statistical selection. Uniqueness determines whether accurate solvers must agree on coefficients (Tibshirani, 2013); support recovery needs assumptions on the predictors (Zhao and Yu, 2006). Earlier coordinate-descent work supplies both convergence conditions (Tseng, 2001) and a regression-specific implementation (Wu and Lange, 2008). Elastic net provides context for correlated predictors (Zou and Hastie, 2005), while the experiments retain the squared-loss Lasso objective and the same three optimization algorithms.

Table 1. Literature organized by mechanism and question.

Mechanism / question	Reference	Key idea
Penalty geometry	Tibshirani (1996)	ℓ_1
Thresholding	Daubechies et al. (2004)	$\mathscr{S}_\tau$
Extrapolation	Beck and Teboulle (2009)	$O(k^{-2})$
Scalar minimization	Wu and Lange (2008)	$r^{(j)}$
Regression paths	Friedman et al. (2010)	Warm starts
Convergence conditions	Tseng (2001)	Separability
Coordinate choice	Wright (2015)	Cyclic / random
Curvature-based steps	Kim et al. (2007)	Barrier + Newton
Splitting	Boyd et al. (2011)	ADMM
Variable elimination	Ndiaye et al. (2017)	Safe region

2 The Lasso Training Problem

2.1 Objective and preprocessing

Write $X \in \mathbb{R}^{n\times d}$ for the training matrix, $y \in \mathbb{R}^n$ for centered responses, and $w \in \mathbb{R}^d$ for coefficients. With $\lambda > 0$ controlling regularization, the squared-loss Lasso objective is

$$F(w) = g(w) + h(w) = \frac{1}{2n}\|Xw - y\|_2^2 + \lambda\|w\|_1, \qquad \lambda > 0. \tag{1}$$

The loss measures training fit, while the penalty encourages sparse coefficients. Every implementation uses Eq. (1). A different loss normalization would change both the penalty scale and the gradient, so the objective, threshold, and step size must use the same convention.

Features use their training means and population standard deviations; responses use their training mean. Test observations use the same statistics, and predictions restore the response mean. Constant training features are removed. Centering fits an unpenalized intercept without using test information.

Feature units affect coefficient magnitudes and hence the penalty. Standardization is therefore part of the model specification, and every solver receives the same transformed inputs. For comparisons with synthetic ground truth, fitted coefficients are converted back to the original feature units.

2.2 Convex formulations

The constrained view of Lasso minimizes the loss over a norm ball:

$$\min_{w} g(w) \quad \text{subject to} \quad \|w\|_1 \le t, \qquad t \ge 0. \tag{2}$$

For any minimizer $w^\star$ of Eq. (1), choosing $t = \|w^\star\|_1$ makes it optimal for Eq. (2). A feasible point with smaller loss would also have a smaller penalized objective. Conversely, for $t > 0$, strict feasibility at $w = 0$ yields a constraint multiplier $\lambda \ge 0$ and a corresponding penalized problem. The correspondence need not be one-to-one (Tibshirani, 1996; Boyd and Vandenberghe, 2004).

Introducing $u \in \mathbb{R}^d$ gives an exact reformulation of the penalized model:

$$\min_{w,u} \frac{1}{2n}\|Xw - y\|_2^2 + \lambda \sum_{j=1}^{d} u_j \quad \text{subject to} \quad -u \le w \le u. \tag{3}$$

The inequalities act coordinate by coordinate. Since $\lambda > 0$, every optimum has $u_j = |w_j|$. A convex quadratic objective and affine inequalities make Eq. (3) a convex quadratic program (Boyd and Vandenberghe, 2004). This formulation connects the norm-ball geometry with standard convex modeling; the implemented solvers work directly with the original objective in Eq. (1).

2.3 Convexity, existence, and optimality

The loss gradient supplies the fitting step in ISTA and FISTA. The Hessian is positive semidefinite because $a^\top X^\top X a = \|Xa\|_2^2 \ge 0$ for every a, which proves convexity:

$$\nabla g(w) = \frac{X^\top (Xw - y)}{n}, \qquad \nabla^2 g(w) = \frac{X^\top X}{n} \succeq 0. \tag{4}$$

Hence g is convex. The norm $\|w\|_1$ is also convex, so F is convex. Since $\lambda > 0$, $F(w)$ grows without bound as $\|w\|_2$ grows without bound; a minimizer exists. Full column rank of X is a sufficient condition for strong convexity and a unique coefficient solution. Convexity alone does not guarantee uniqueness, especially in problems with more features than samples.

Full column rank is sufficient rather than necessary for uniqueness. Tibshirani (2013) examines unique and nonunique Lasso solutions, including problems with more predictors than observations. When coefficients are not unique, optimal solutions can have different active sets despite agreeing on fitted values. This distinction matters when using coefficient agreement to validate a solver.

The subgradient of $|u|$ at zero is $[-1, 1]$. Applying this rule coordinate by coordinate gives the optimality condition below: the loss gradient must be balanced by a subgradient of the penalty.

$$0 \in \nabla g(w^\star) + \lambda \partial \|w^\star\|_1. \tag{5}$$

The QP in Eq. (3) is strictly feasible at $w = 0$, $u = \mathbf{1}$, regardless of the column rank of X. Its Karush–Kuhn–Tucker (KKT) conditions characterize global optima and reduce to Eq. (5) after eliminating the auxiliary variables and multipliers (Boyd and Vandenberghe, 2004).

The two cases are $\nabla_j g(w^\star) = -\lambda \operatorname{sign}(w_j^\star)$ for $w_j^\star \ne 0$ and $|\nabla_j g(w^\star)| \le \lambda$ for $w_j^\star = 0$. The interval in the second case permits exact zeros. The same optimality condition gives a stationarity check that applies to all three solvers at a common accuracy (Parikh and Boyd, 2014).

2.4 Why the penalty produces sparse coefficients

Lasso and ridge both shrink coefficients: ridge uses a squared ℓ_2 penalty, whereas Lasso uses ℓ_1. In two dimensions, the ℓ_1 constraint has corners on the coordinate axes, where contact with a loss contour can give a zero coefficient (Tibshirani, 1996). The geometry explains sparsity; recovery of the generating variables still needs to be checked against known support.

Selection also depends on the feature relationships. Zhao and Yu (2006) links model-selection consistency to an irrepresentable condition involving the predictor covariance: sparsity alone is not enough for correct recovery. This is an asymptotic statistical result, whereas the present experiment records support on three finite synthetic instances. Convergence to a minimizer and recovery of the generating variables are separate properties and require different forms of evidence.

An orthogonal design gives a direct algebraic example. If $X^\top X/n = I$, the quadratic loss separates into scalar terms. Writing $v = X^\top y/n$, each coefficient has the following closed-form solution:

$$w^\star = \mathscr{S}_\lambda(v), \qquad \mathscr{S}_\tau(v)_j = \operatorname{sign}(v_j)\max\{|v_j| - \tau, 0\}. \tag{6}$$

For example, $v = (1.2, -0.4, 0.1)$ and $\lambda = 0.3$ give $w^\star = (0.9, -0.1, 0)$. The zero interval in Figure 1 removes small coefficients, while the outer segments shrink the retained ones. This analytic example explains why stronger regularization can reduce selected variables without improving test error.

The same operation appears in all three solvers. ISTA and FISTA threshold a vector after a gradient step; coordinate descent thresholds one coefficient within an exact scalar minimization. Each update follows from the objective rather than treating sparsity as a separate selection step.

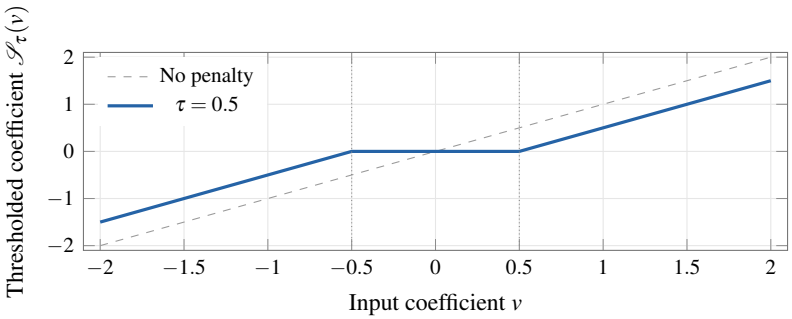


Figure 1. Soft thresholding ($\tau = 0.5$): zeros inside $[-\tau, \tau]$; shrinkage outside.

2.5 Related penalties: changing the model

The same squared loss can be paired with different penalties (Table 2). Ridge adds quadratic shrinkage to stabilize estimates under nonorthogonal predictors (Hoerl and Kennard, 1970). Its smooth penalty generally retains a dense coefficient vector. Elastic net combines sparsity with quadratic shrinkage and encourages grouped selection of correlated predictors (Zou and Hastie, 2005). These choices change the estimator, even if the optimization algorithm stays the same.

Table 2. Related convex penalties; all use the loss $g(w)$ in Eq. (1).

Model	Penalty	Main distinction
Ridge	$\frac{\alpha}{2}\|w\|_2^2$	Quadratic shrinkage
Lasso	$\lambda\|w\|_1$	Uniform thresholds
Elastic net	$\lambda\|w\|_1 + \frac{\alpha}{2}\|w\|_2^2$	Grouping
Adaptive Lasso	$\lambda\sum_j \omega_j\|w_j\|$	Weighted thresholds

$\alpha, \lambda > 0$; fixed $\omega_j > 0$.

Adaptive Lasso uses weights estimated from an initial fit (Zou, 2006). Once finite positive weights are fixed, the objective remains convex and its proximal step uses threshold $\eta\lambda\omega_j$ for coordinate j. The weights can distinguish coefficients that the initial fit suggests are large or small. Its statistical guarantees require assumptions on the initial estimator, design, and penalty sequence; they do not ensure that the fitted support matches the generating variables in any given finite dataset.

This comparison separates two questions. Changing a penalty asks which model gives an appropriate prediction or selection trade-off; changing a solver asks how to compute a solution to a specified objective. To isolate computation, our benchmark fixes the unweighted Lasso penalty and preprocessing before comparing the three implemented optimization algorithms.

3 A Simple Comparison of Update Mechanisms

Table 3 compares the evaluation point, update, and dense cost per round. ISTA uses the current iterate, FISTA uses an extrapolated iterate, and coordinate descent uses the latest partial residual. A coordinate-descent round means a complete scan of all d variables.

Table 3. Update mechanisms.

Method	Input	Update	Cost
ISTA	w^k	$\mathscr{S}_{\lambda/L}(w^k - \nabla g(w^k)/L)$	$O(nd)$
FISTA	z^k	$\mathscr{S}_{\lambda/L}(z^k - \nabla g(z^k)/L)$	$O(nd)$
Cyclic CD	$r^{(j)}$	Eq. (15)	$O(nd)$

$\eta = 1/L$; CD: d coordinates / sweep.

3.1 ISTA

The proximal subproblem separates across coordinates. The ℓ_1 penalty can therefore be handled through scalar minimizations with closed-form solutions, giving the following vector operation:

$$\arg\min_u \left\{ \frac{1}{2}\|u - v\|_2^2 + \tau\|u\|_1 \right\} = \mathscr{S}_\tau(v). \tag{7}$$

For a positive scalar solution, differentiation gives $u = v - \tau$; for a negative solution, $u = v + \tau$. If $|v| \leq \tau$, zero satisfies the subgradient condition. The three cases give soft thresholding in Eq. (6) (Parikh and Boyd, 2014), including an interval on which the output is exactly zero.

The gradient of g is Lipschitz continuous with constant $L = \|X\|_2^2/n$, where $\|X\|_2$ is the spectral norm. For $0 < \eta \leq 1/L$, the smooth loss has a quadratic upper bound at the current iterate. Minimizing this bound together with the penalty gives the next coefficient vector through the update

$$w^{k+1} = \mathscr{S}_{\eta\lambda}\left(w^k - \eta\nabla g(w^k)\right). \tag{8}$$

ISTA takes a gradient step for fitting and a thresholding step for the penalty. Daubechies et al. (2004) developed iterative thresholding for linear inverse problems with sparsity constraints. Here the same mechanism solves the finite-dimensional squared-loss regression objective.

ISTA is simple to implement and has a nonincreasing objective with the stated step size. Its dense work is dominated by Xw and $X^\top(Xw - y)$, without storing a full $d \times d$ matrix. Its basic objective-error bound is slower than FISTA's; choosing a smaller step can further increase the update count.

3.2 FISTA

FISTA applies the proximal step to an extrapolated point (Beck and Teboulle, 2009). Initialize $z^1 = w^0$ and $t_1 = 1$, then use $\eta = 1/L$. The sequence t_k controls the contribution of the previous direction. Each round computes the coefficients before the next extrapolation:

$$w^k = \mathscr{S}_{\eta\lambda}\left(z^k - \eta\nabla g(z^k)\right), \tag{9}$$

$$t_{k+1} = \frac{1 + \sqrt{1 + 4t_k^2}}{2}, \tag{10}$$

$$z^{k+1} = w^k + \frac{t_k - 1}{t_{k+1}}(w^k - w^{k-1}), \qquad k \geq 1. \tag{11}$$

The extrapolation uses the recent direction of movement. It requires extra vector operations and a previous iterate, while the dominant matrix-vector products remain the same as those in ISTA.

For convex composite objectives with a Lipschitz-continuous smooth gradient, an existing minimizer, exact proximal updates, and step size $1/L$, the standard bounds are

$$F(w^k) - F^\star \leq \frac{L\|w^0 - w^\star\|_2^2}{2k} \qquad \text{(ISTA)}, \tag{12}$$

$$F(w^k) - F^\star \leq \frac{2L\|w^0 - w^\star\|_2^2}{(k+1)^2} \qquad \text{(FISTA)}. \tag{13}$$

These are objective-error bounds, not coefficient-error or elapsed-time bounds. They describe a worst-case guarantee and do not imply that every FISTA iterate improves on ISTA. Standard FISTA can have a nonmonotone objective. Our comparison uses this standard version without restart or monotonicity modifications, so that the algorithm being evaluated is unambiguous.

3.3 Cyclic coordinate descent

Coordinate descent fixes all coefficients except one and minimizes the resulting scalar objective. Write x_j for column j of X and use the latest available coefficients to define

$$r^{(j)} = y - \sum_{\ell \neq j} x_\ell w_\ell. \tag{14}$$

The scalar objective contains a quadratic loss and an absolute-value penalty. Its solution uses soft thresholding, divided by the squared norm of the feature column to account for its scale:

$$\begin{aligned} &\min_u \frac{1}{2n} \|r^{(j)} - x_j u\|_2^2 + \lambda |u|, \\ &w_j \leftarrow \frac{\mathscr{S}_\lambda(x_j^\top r^{(j)}/n)}{\|x_j\|_2^2/n}. \end{aligned} \tag{15}$$

The denominator is one for centered, unit-variance columns. The general formula retains the dependence on feature scaling, as in the regression coordinate update of Friedman et al. (2010).

Maintain $r = y - Xw$ throughout the sweep. Before updating coordinate j, form $r^{(j)} = r + x_j w_j$; afterwards, set $r = r^{(j)} - x_j w_j$. Each coordinate costs $O(n)$ for dense data, and a sweep costs $O(nd)$. Updating the residual after every coefficient change avoids repeated full predictions.

Wu and Lange (2008) studies cyclic coordinate descent for squared-error regression with a Lasso penalty. That setting matches our scalar updates; the paper's absolute-error and grouped extensions are outside this report. It provides a second regression-specific source alongside the pathwise implementation of Friedman et al. (2010) for regularized least-squares regression.

Exact coordinate minimization makes the objective nonincreasing and needs no global gradient step size. Update order and problem assumptions still matter. Wright (2015) distinguishes cyclic, randomized, smooth, and regularized methods. A randomized rate cannot be transferred directly to this cyclic Lasso implementation; the rate entry is therefore left unspecified.

Tseng (2001) analyzes block coordinate descent when the nonsmooth component is separable and further regularity conditions hold. In Lasso, $\|w\|_1 = \sum_j |w_j|$ supplies this separability. The result explains why the structure of the penalty matters; it does not justify convergence claims for arbitrary nonsmooth objectives or an unconditional rate for our implementation.

4 The Wider Solver Landscape

The three implemented methods use either a full vector or one coordinate at a time. Other approaches use a solution path, curvature information, or an equality-constrained split. Table 4 compares their operations and additional work; their iteration counts refer to different optimization subproblems.

Table 4. Solver families and the work they expose.

Family	Main operation	Extra work / state
ISTA / FISTA	Vector threshold	Matrix-vector products
Coordinate descent	Scalar minimization	Residual; coordinate scan
LARS–Lasso	Path segment	Active-set linear algebra
Interior point	Barrier Newton step	Linear-system iterations
ADMM	Split subproblems	Linear solve; dual state

4.1 Paths, curvature, and splitting

LARS with the Lasso modification follows the solution path as the penalty changes (Efron et al., 2004). Pathwise coordinate descent instead solves successive penalties with warm starts (Friedman et al., 2010). Reusing a nearby solution can reduce subsequent work, but path construction and a cold solve at one penalty have different costs. Our fixed-penalty runs start at zero; the separate penalty study is not a benchmark of either of these methods for constructing solution paths.

Kim et al. (2007) introduces auxiliary bounds on coefficients to express the absolute-value penalty as a linear term in a quadratic program. A logarithmic barrier gives approximate Newton steps, computed with preconditioned conjugate gradients in their specialized implementation. This replaces many simple updates with more involved linear algebra. Its value depends on structure and accuracy; paper timings do not describe the runtime of a generic quadratic-program solver.

ADMM separates the quadratic loss and the ℓ_1 penalty using an equality constraint (Boyd et al., 2011). Its Lasso iteration alternates a regularized linear solve, soft thresholding, and a dual update. The algorithmic penalty is distinct from the model's λ; fixing it can permit factorization reuse. Consequently, setup and solve costs both matter when comparing this splitting method with the matrix-vector operations used by the two proximal-gradient implementations in this report.

4.2 Reducing repeated or unnecessary work

Acceleration can create oscillations, so momentum management is a separate design choice. O'Donoghue and Candès (2015) proposes restarting based on objective or gradient criteria and discusses a FISTA application. Resetting extrapolation may help when its direction is unproductive, but frequent restarting can also slow progress. The paper suggests an extension to standard FISTA; residual oscillations do not establish that restarting would reduce our measured runtime.

Safe screening removes variables only when a valid test proves that their optimal coefficients are zero (Ndiaye et al., 2017). Gap Safe rules form a region containing the dual optimum from a dual-feasible point and a primal–dual gap. A shrinking region can exclude more variables as optimization progresses. Screening adds certificate calculations but reduces the optimization dimension; its usefulness depends on the feature count and the amount of work that can be removed.

The gap is also an accuracy certificate: a primal objective minus a feasible dual lower bound bounds objective suboptimality. This differs from our stationarity-residual threshold. Neither criterion identifies the generating variables merely by certifying optimization progress. The broader literature therefore suggests three distinct comparisons: update mechanisms, reusable or removable work, and the certificate used to declare a solution accurate enough for the stated training problem.

5 Strengths, Weaknesses, and Comparison Limits

Table 5 records monotonicity and objective-error bounds. ISTA and FISTA assume a convex objective, a Lipschitz-continuous smooth gradient, exact proximal updates, and a valid step size (Beck and Teboulle, 2009). No universal rate is assigned to cyclic coordinate descent.

Table 5. Monotonicity and objective-error bounds.

Method	$F_{k+1} \le F_k$	Bound	Step	State
ISTA	✓	$O(k^{-1})$	$1/L$	—
FISTA	—	$O(k^{-2})$	$1/L$	z,t
Cyclic CD	✓	—	—	r

$F_k = F(w^k)$; ✓: monotone; —: no claim / n.a.

A scalar coordinate update is smaller than a vector update, so we compare complete CD sweeps with ISTA/FISTA rounds and also report time. Dense work is $O(nd)$ per round, but loops and matrix-vector kernels have different overheads. Accurate solvers at a fixed penalty should reach similar objectives; coefficient agreement additionally depends on uniqueness.

The penalty controls the fit–sparsity trade-off. A test-error difference at a fixed penalty should first be checked against stopping accuracy and preprocessing. Changing the penalty studies the model rather than the solver, and a sparse fit alone does not establish recovery of the generating variables.

6 Data and Experimental Protocol

6.1 Data

The original benchmark reads the saved inputs and fixed splits in Table 6; controlled extensions are specified separately in Section 7.7. Features and responses retain their original units, and each observation has a recorded split assignment. Centering and scaling use only the selected training rows, so all solvers receive identical transformed data.

Table 6. Dataset sizes per instance.

Input	Instances	N	d	Train / test
Synthetic sparse regression	3	300	50	240 / 60
Diabetes regression	1	442	10	353 / 89

Synthetic features are independent standard-normal draws. The first five true coefficients are $(2, -1.5, 1, -2, 1.5)$, and the remaining 45 are zero. Responses satisfy $y = Xw_{\text{true}} + \varepsilon$ with normal noise of standard deviation 0.5. Seeds 0, 1, and 2 define three independently generated instances.

Diabetes contains 442 observations, ten baseline predictors, and a continuous measure of disease progression one year later (Efron et al., 2004). We use the raw files distributed with scikit-learn 1.9.1 and standardize them on the training split (scikit-learn developers, 2026). Unlike the synthetic instances, this dataset has no known generating coefficients or correct support set.

A separate NumPy generator permutes row indices with the recorded split seed. The first $\lceil 0.2N \rceil$ rows form the test set. Split seeds are 0, 1, and 2 for the synthetic instances and zero for Diabetes. Saved split assignments and input hashes identify the observations used in the experiments.

6.2 Common settings and stopping check

Table 7 gives the settings fixed before measurement. The penalty scale comes from the transformed training inputs. A zero vector becomes optimal at the largest absolute feature–response inner product divided by sample size. This defines the training-dependent reference value

$$\lambda_{\max} = \frac{\|X^\top y\|_\infty}{n}. \tag{16}$$

Eq. (5) shows that $w = 0$ is optimal when $\lambda \geq \lambda_{\max}$. The main comparison uses $\lambda = 0.1\lambda_{\max}$. This choice provides a nontrivial sparse fit; it is not claimed to minimize prediction error.

Table 7. Experimental settings.

Setting	Value
Initialization	$w^0 = 0$
Penalty	$\lambda/\lambda_{\max} = 0.1$
ISTA / FISTA step	$\eta = 1/L, \quad L = \|X\|_2^2/n$
CD order	$j = 1, \ldots, d$
Stopping	$R \leq 10^{-6}$
Maximum rounds	10^4
Timing	5 repeats; median
Penalty ratios	$\{0.01, 0.1, 0.5, 1\}$ (CD)

A common stopping check is needed because small parameter changes do not necessarily imply comparable optimization accuracy. Define the proximal-gradient mapping and normalized residual

$$G_L(w) = L\left[w - \mathscr{S}_{\lambda/L}\left(w - \frac{\nabla g(w)}{L}\right)\right], \tag{17}$$

$$R(w) = \frac{\|G_L(w)\|_\infty}{\max\{1, \|X^\top y/n\|_\infty\}}. \tag{18}$$

The mapping is zero exactly when the optimality condition holds (Parikh and Boyd, 2014). Its norm measures stationarity; it does not certify an upper bound on objective error. We evaluate it after every vector update or CD sweep, at the returned coefficient vector rather than the FISTA extrapolation point. A run that reaches its iteration limit before meeting the threshold is marked as unconverged.

6.3 Comparison and measurement

At a fixed penalty, we compare objective trajectories, stationarity residuals, complete rounds, and elapsed time. Test MSE and nonzero counts describe the resulting fit. They answer a different question from the computation required to reach the prescribed optimization tolerance.

Timing includes algorithm-specific preparation, objective recording, and the common residual check. Loading, preprocessing, the shared spectral-norm calculation, and plotting are excluded. No solver uses a precomputed Gram matrix or screening. Five timing repetitions yield one median per instance; the synthetic summary reports variation over three instances, while Diabetes uses one fixed split.

For the penalty study, CD solves four prespecified values to the same tolerance. We report test MSE and coefficients satisfying $|w_j| > 10^{-6}$ in standardized units. The test scores describe these settings; they are not used to select a penalty and then claim an unbiased estimate of its prediction error.

6.4 Minimal implementation validation

We first check an orthogonal design against Eq. (6), then compare synthetic seed 0 with a tighter scikit-learn solve. The reference uses centered inputs, `alpha` $= \lambda$, and no fitted intercept, matching the documented objective (scikit-learn developers, 2026). Comparing coefficients, objective, and stationarity residual checks numerical agreement before the timed runs.

The orthogonal case checks thresholding and normalization against a closed-form solution. The reference case checks a nonorthogonal design against another implementation. Both checks precede the timed runs and address numerical correctness rather than providing evidence about solver speed.

7 Experimental Results and Discussion

7.1 Validation and execution

Both implementation checks passed. The largest absolute coefficient error on the orthogonal instance was 1.12×10^{-15}. On synthetic seed 0, relative objective differences from the tighter scikit-learn reference were at most 1.57×10^{-12}; the reference residual was 1.54×10^{-14}. These checks support the implemented normalization and updates, but do not certify a general solver ranking.

All 60 main timing runs (four instances, three methods, five repetitions) and all 16 regularization runs reached $R \leq 10^{-6}$. Execution used Python 3.12.14, NumPy 2.3.5, scikit-learn 1.9.1, and Matplotlib 3.11.2 on an arm64 macOS host, with available numerical thread pools limited to one thread. Raw timings, histories, coefficients, input hashes, and environment versions are retained.

7.2 Efficiency at a common tolerance

Table 8. Solver comparison at $\lambda/\lambda_{\max} = 0.1$.

Method	Time (ms)	Rounds	Final F	R (10^{-6})
Synthetic: three instances				
ISTA	**0.389** $\pm$ 0.013	24.7 $\pm$ 0.6	1.800 $\pm$ 0.174	0.71 $\pm$ 0.16
FISTA	0.519 $\pm$ 0.035	30.7 $\pm$ 2.3	1.800 $\pm$ 0.174	0.53 $\pm$ 0.30
CD	0.596 $\pm$ 0.010	5.0	1.800 $\pm$ 0.174	0.14 $\pm$ 0.17
Diabetes: fixed split				
ISTA	1.419	92.0	1812.178	0.99
FISTA	1.437	85.0	1812.178	0.31
CD	**0.530**	14.0	1812.178	0.40

$\mu \pm s$: three instances; t = median(5); **bold**: min. t.

On the synthetic instances, ISTA used 24–25 updates, FISTA used 28–32, and CD used five sweeps. ISTA had the shortest measured time on all three instances. CD completed fewer sweeps but each sweep made 50 sequential scalar updates in a Python loop. ISTA and FISTA used NumPy matrix-vector operations. These timings are consistent with loop overhead affecting CD at this size, although loop overhead was not measured by timing the individual operations separately.

On Diabetes, CD required 14 sweeps and a median of 0.530 ms, compared with 92 updates and 1.419 ms for ISTA and 85 updates and 1.437 ms for FISTA. CD was fastest in this setting. ISTA and FISTA timing ranges overlapped: 1.326–1.438 ms and 1.357–1.450 ms, respectively. Their small median difference does not support a robust speed distinction between these vector methods.

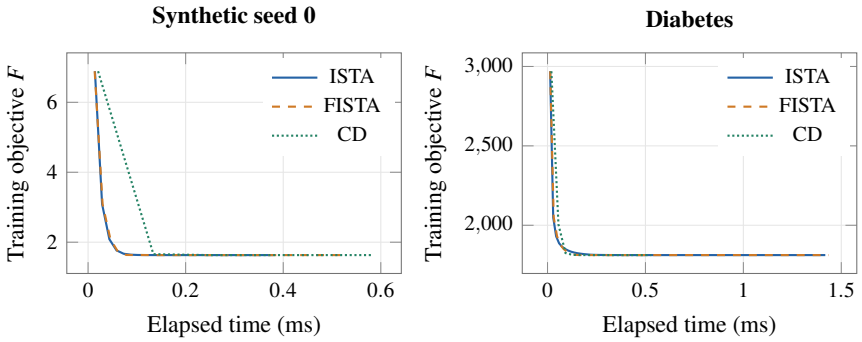


Figure 2. Objective versus time; median-time repetition.

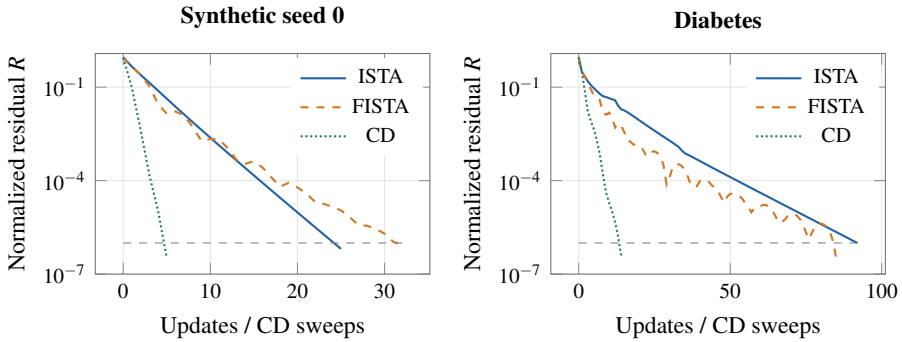


Figure 3. Residual versus rounds. Dashed line: $R = 10^{-6}$.

Figure 3 shows oscillations in FISTA's residual and a smoother decrease for ISTA. Acceleration did not reduce the synthetic update count at this threshold. Eq. (13) bounds worst-case objective error, whereas the stopping rule concerns an instance's proximal-gradient residual. Initial objective reduction alone therefore cannot determine time to the declared tolerance (Figure 2). These observations concern standard FISTA; restart variants need a separate comparison.

The shared spectral-norm calculation took 0.278–0.714 ms for synthetic instances and 3.070 ms for Diabetes. It is excluded from Table 8 but would matter in an end-to-end application at these sizes. CD does not need L for its scalar update; the common residual check does. These timings depend on the recorded input sizes, the solver and monitoring implementations, and the machine.

7.3 What the timing repetitions establish

Figure 4 retains all five measurements rather than displaying only medians. ISTA's range lies below the other methods' ranges on each synthetic instance, while CD's range lies below both vector methods on Diabetes. The ISTA/FISTA overlap on Diabetes supports treating their small median difference cautiously. No significance test is claimed from five short timing repetitions.

Rounds and time give different rankings. Five synthetic CD sweeps contain $5 \times 50 = 250$ scalar updates; a 25-round ISTA run contains 25 vector updates. Both have dense work of order nd per

round, but scalar loops and matrix-vector kernels differ. This is consistent with an implementation cost explanation, although the experiment does not isolate the overhead of each operation.

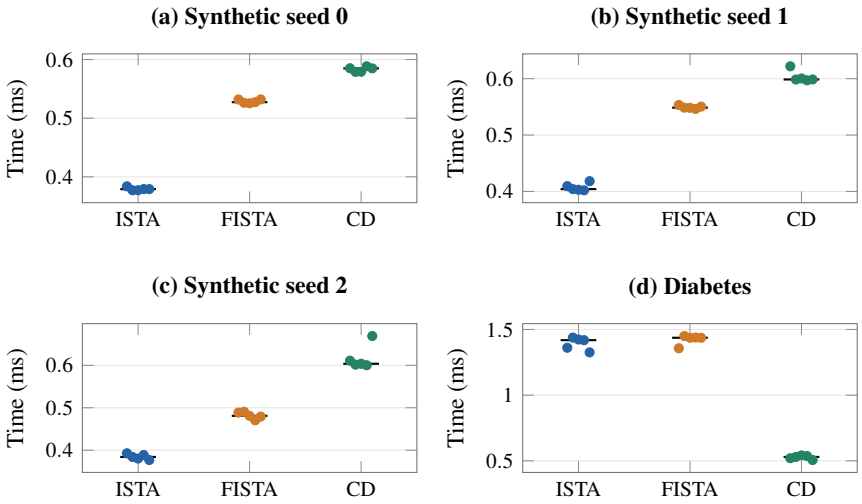


Figure 4. Timing repetitions ($n = 5$). Black bars: medians.

7.4 Prediction and sparsity at the fixed penalty

Table 9. Prediction at $\lambda/\lambda_{\max} = 0.1$.

Method	Test MSE	Nonzero
Synthetic: three instances		
ISTA	0.396 ± 0.139	5
FISTA	0.396 ± 0.139	5
CD	0.396 ± 0.139	5
Diabetes: fixed split		
ISTA	2966.935	5
FISTA	2966.936	5
CD	2966.937	5

$\mu \pm s$: three instances; nonzero: $|w_j| > 10^{-6}$.

The methods produced nearly identical objectives and prediction errors, retaining five coefficients on every instance. Diabetes MSE differences are below 0.002 on a scale near 2967 and do not establish a predictive advantage for any optimizer. Synthetic fits selected the generating variables, with magnitudes shrunk toward zero. The selected count on Diabetes cannot be assessed as recovery accuracy because its true support and generating coefficients are unknown.

7.5 Support recovery and shrinkage are different

Figure 5 separates selection from magnitude estimation. All three synthetic CD fits retain the five generating coefficients and set the 45 inactive coefficients to zero. Mean active estimates are $(1.801, -1.260, 0.736, -1.810, 1.287)$, closer to zero than the generating values. Their mean absolute error is 0.221 ± 0.025 across instances. Selecting the correct variables therefore does not remove the shrinkage error in their fitted coefficients, even when every active variable is retained.

The Diabetes solution retains sex, bmi, bp, s3, and s5 on the fixed split. Signs and magnitudes describe a standardized fit; they do not establish causal effects or stable selection across splits. The synthetic panel compares fitted coefficients with the generating values in original units. The Diabetes panel compares predictors through their fitted standardized coefficients.

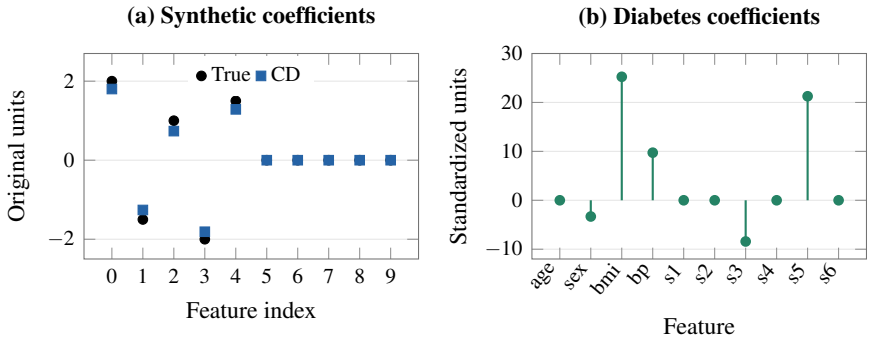


Figure 5. CD coefficients. Left: synthetic mean ± SD (original units); right: Diabetes (standardized units).

7.6 Regularization effects

Table 10. Regularization study (CD).

$\lambda/\lambda_{\max}$	Test MSE	Nonzero
Synthetic: three instances		
0.01	0.236 ± 0.033	22.3 ± 3.5
0.1	0.396 ± 0.139	5.0
0.5	5.506 ± 1.021	4.3 ± 0.6
1	11.945 ± 1.311	0.0
Diabetes: fixed split		
0.01	2912.647	8.0
0.1	2966.937	5.0
0.5	3837.820	2.0
1	5893.048	0.0

$\mu \pm s$: three instances; nonzero: $|w_j| > 10^{-6}$.

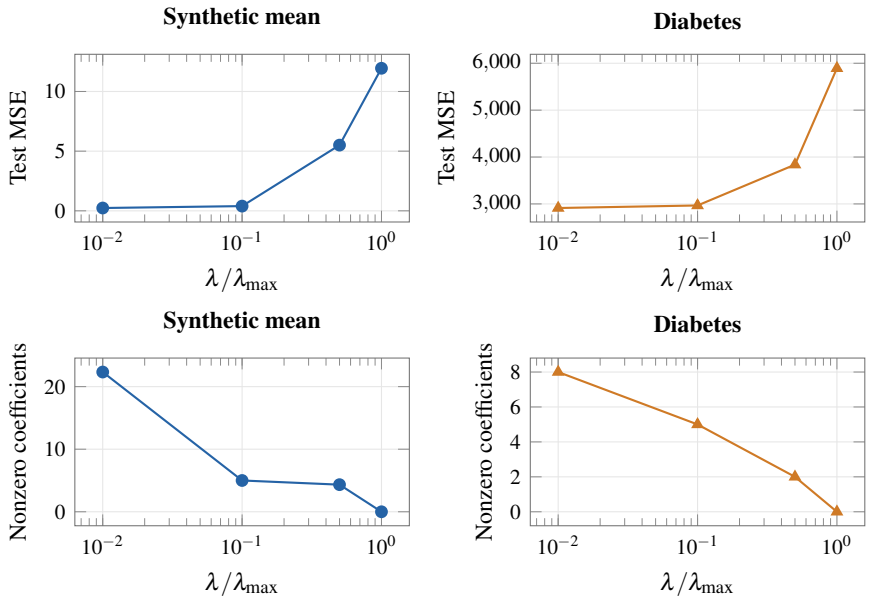


Figure 6. Test MSE and selected variables versus penalty; synthetic means over three instances.

Increasing the penalty reduced selected variables in these cases. For synthetic data, the mean count fell from 22.3 at ratio 0.01 to five at ratio 0.1, while mean test MSE rose from 0.2360 to 0.3960. The

stronger penalty produced a smaller model without the lowest average prediction error. Individual errors were not strictly monotone: seed 1 had slightly lower MSE at 0.1 than at 0.01. At ratio 0.5, mean MSE rose to 5.5058, consistent with increased shrinkage bias.

For Diabetes, the selected count decreased from eight to five to two as the ratio increased from 0.01 to 0.1 to 0.5, and test MSE increased. At ratio 1, both datasets returned zero coefficients at initialization, as predicted by the optimality condition. Predictions then equal the training response mean. This fit–sparsity trade-off concerns the model and penalty, not superiority of CD as a predictor.

7.7 Controlled correlation and dimension studies

The original benchmark is small and mostly well conditioned. To test whether its ordering persists, we vary within-block feature correlation and then feature count. The correlation study uses 600 observations and 60 features in blocks of ten. For feature j in block b, independent standard-normal draws give

$$x_j = \sqrt{\rho}\, z_b + \sqrt{1-\rho}\, z_j, \qquad \rho \in \{0, 0.5, 0.9, 0.99\}. \tag{19}$$

The first five coefficients are $(2, -1.5, 1, -2, 1.5)$, and noise has standard deviation 0.5. Seeds 10–12 share underlying random draws, noise, and a 480/120 split across correlation levels. The dimension study uses independent features, 1,000 observations, an 800/200 split, and $d \in \{50, 100, 300, 600\}$. Within each seed, smaller designs use prefixes of the same 600-column matrix and the same response vector.

Both studies retain zero initialization, $\lambda = 0.1\lambda_{\max}$, $R \le 10^{-6}$, and the 10,000-round limit. Each instance has five interleaved timings; reported means and sample standard deviations describe three instance medians. Solver time includes monitoring but excludes the shared spectral calculation. Thread settings are requested before importing the numerical libraries. These timings form a separate run and are not pooled with the original benchmark.

All 360 timed solves converge. The largest relative objective spread between solvers on the same problem is 8.61×10^{-11}. This check supports comparison at the stated stationarity tolerance; it does not make R an objective-error bound. Actual inputs, splits, code snapshots, settings, and complete traces are retained with the new results.

Table 11. Correlation study: condition number and solver time (ms).

ρ	κ	ISTA	FISTA	Cyclic CD
0	4.1	0.360 ± 0.010	0.521 ± 0.043	0.762 ± 0.107
0.5	28.8	3.236 ± 0.223	3.782 ± 0.208	3.318 ± 0.321
0.9	230.3	21.667 ± 1.026	16.876 ± 1.160	16.443 ± 1.622
0.99	2492.8	154.599 ± 10.491	19.862 ± 18.153	62.507 ± 8.707

$\mu \pm s$: three instance medians; κ: mean full-Gram condition number.

At $\rho = 0$, ISTA takes 0.360 ± 0.010 ms. At $\rho = 0.99$, its mean time rises to 154.599 ms, while FISTA takes 19.862 ± 18.153 ms and CD takes 62.507 ± 8.707 ms. Acceleration has a measured advantage in this highly correlated setting, although the three FISTA instances vary widely. At $\rho = 0.5$, the ISTA and CD means are close; the measurements do not establish a precise ranking between them.

The mean condition number $\kappa(X^\top X/n)$ grows from 4.1 to 2492.8. Greater correlation creates weak-curvature directions, which can slow fixed-step progress. The selected support and penalty scale also change, so this full-design condition number is a diagnostic rather than a complete prediction of solver time. The recorded $\rho = 0.99$ trace shows FISTA reaching the threshold sooner despite residual oscillations; smooth progress and fast stopping are different properties.

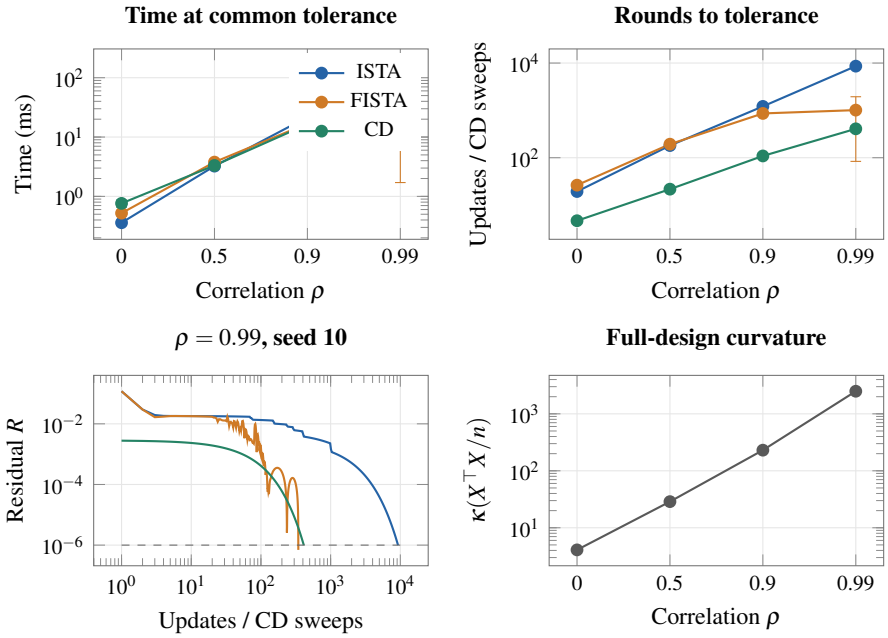


Figure 7. Correlation, solver cost, and curvature. Error bars: sample SD across three instance medians. The residual panel uses seed 10; dashed line: $R = 10^{-6}$. Long traces are sampled for display.

7.8 Feature count and work per round

Figure 8 fixes the sample count while increasing the feature count. CD uses four complete sweeps for all twelve instances, but its mean time rises from 0.651 ms at $d = 50$ to 6.972 ms at $d = 600$. A sweep updates every coordinate, so a constant sweep count still entails more scalar updates. Python-loop and matrix-vector costs remain part of these implementations rather than isolated causes in a profiling study.

ISTA is fastest at every tested width: its mean time increases from 0.341 to 2.078 ms, while its mean round count increases from 15.0 to 43.3. FISTA takes 0.498 to 2.795 ms and more rounds under this residual rule. The width study therefore preserves the ordering of the original independent-feature experiment, whereas strong correlation changes it. This comparison does not cover sparse matrices, $d > n$, or compiled coordinate-descent implementations.

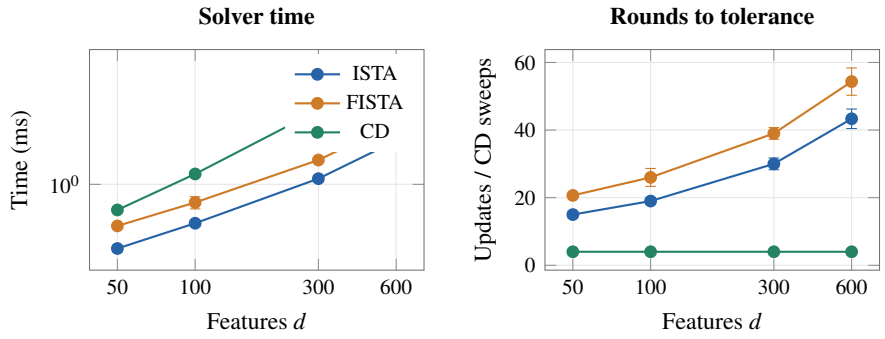


Figure 8. Feature-count study with 800 training observations. Means and sample SD across three instance medians; five timings per instance. Shared spectral setup is excluded.

7.9 Coefficient paths and fitting error

The original four-penalty study is extended to 34 pre-specified values between $0.01\lambda_{\max}$ and $\lambda_{\max}$ on synthetic seed 0 and the fixed Diabetes split. CD starts from zero for each fit and reaches the

same tolerance. Figure 9 records the coefficients as well as training and test MSE, without selecting a penalty from test scores.

The synthetic paths show retained coefficients approaching zero as the penalty strengthens; weaker penalties also admit small coefficients on inactive variables. On Diabetes, the five highlighted paths are the predictors selected at the main ratio 0.1, not a known true support. Their magnitudes need not all shrink monotonically: s3 first moves farther below zero before returning toward zero. Unlike the orthogonal formula, correlated predictors are fitted jointly, and the penalty changes their allocation of the shared signal.

Large penalties increase fitting error sharply on both fixed splits. Training MSE can exceed test MSE on a particular split; these curves are neither uncertainty estimates nor a generalization bound. A validation procedure would be needed to select a penalty for prediction. The paths explain how the model changes, while the timing studies compare how different algorithms solve a fixed model.

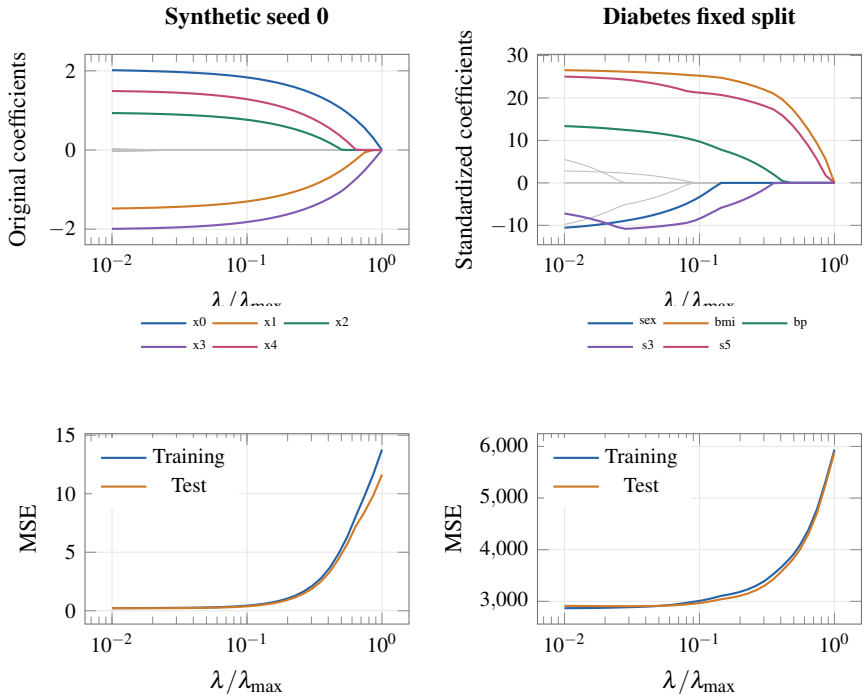


Figure 9. Coefficient and error paths over 34 penalties (CD, cold starts). Colored: true active coefficients in the synthetic case; predictors selected at ratio 0.1 in Diabetes. Gray: remaining coefficients. Each column uses its own response scale.

7.10 A two-dimensional view of the updates

A small quadratic example makes the update mechanisms visible without a timing claim. Set $a = (2,-1)^\top$, $Q = \left(\begin{smallmatrix} 1 & 0.9 \\ 0.9 & 1 \end{smallmatrix}\right)$, and $F_{\text{toy}}(w) = \frac{1}{2}(w-a)^\top Q(w-a) + 0.03\|w\|_1$. All solvers start at zero, and the unique minimizer is $(1.7,-0.7)^\top$.

Figure 10 shows the first 25 vector updates or complete CD sweeps. ISTA advances slowly along the elongated contours; FISTA reaches the optimum's neighborhood and crosses it; CD moves along one coordinate at a time. One CD sweep contains two scalar moves, so the panels illustrate geometry rather than equal-cost speed. The traced implementations' terminal coefficients match the existing solvers exactly on this example.

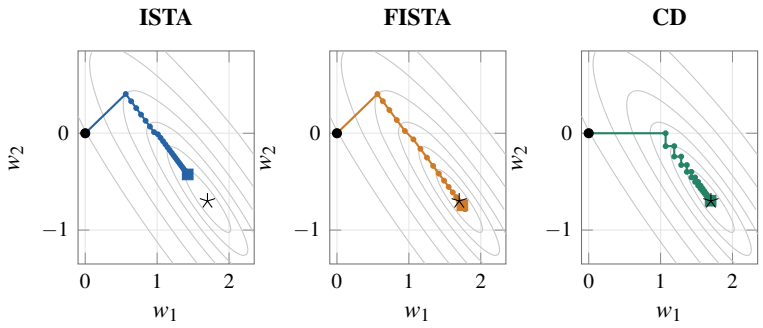


Figure 10. First 25 vector updates / CD sweeps on the same two-dimensional Lasso objective. Gray: contours; dot: start; star: minimizer; colored square: final displayed iterate. Each CD sweep includes two scalar steps.

8 Limitations and Conclusions

Lasso's squared loss and separable ℓ_1 penalty give three simple mechanisms: ISTA combines a gradient step with thresholding, FISTA adds extrapolation, and cyclic CD minimizes one coordinate with a maintained residual. Theory bounds objective error; experiments measure the cost of reaching a shared stationarity tolerance on the recorded synthetic and Diabetes training problems.

ISTA was fastest on the three original synthetic instances, and CD was fastest on the fixed Diabetes split. Standard FISTA's stronger objective-error bound did not produce a time advantage under the common residual rule. Once converged, the solvers gave nearly identical objectives and predictions. Stronger penalties reduced selected variables but usually increased test error in the measured cases.

Strong correlation changes the relative performance of the proximal methods: FISTA is faster on average at $\rho = 0.99$, with large variation across instances. Increasing feature count in independent designs preserves ISTA's advantage. CD uses few sweeps, but their growing coordinate cost prevents sweep count alone from predicting time.

The extensions cover four block-correlation levels and feature counts up to 600, but not all covariance structures, underdetermined designs, or production solvers. Python loops, vectorized operations, and monitoring costs affect the results. Three synthetic instances give limited descriptive variation; timing repetitions are not independent data samples. A single real-data split cannot estimate uncertainty over splits. Test scores were not used to tune the penalty, and a small stationarity residual is not an objective-error certificate. The conclusions apply to the recorded datasets, preprocessing, implementations, and common stopping settings.

Generative AI assistance

Generative AI tools assisted with literature organization, drafting, and experimental code. The reported experiments were executed locally, and citations were checked against primary sources.

References

Amir Beck and Marc Teboulle. 2009. A fast iterative shrinkage-thresholding algorithm for linear inverse problems. *SIAM Journal on Imaging Sciences*, 2(1):183–202.

Stephen Boyd and Lieven Vandenberghe. 2004. Convex Optimization. Cambridge University Press.

Stephen Boyd, Neal Parikh, Eric Chu, Borja Peleato, and Jonathan Eckstein. 2011. Distributed optimization and statistical learning via the alternating direction method of multipliers. *Foundations and Trends in Machine Learning*, 3(1):1–122.

Ingrid Daubechies, Michel Defrise, and Christine De Mol. 2004. An iterative thresholding algorithm for linear inverse problems with a sparsity constraint. *Communications on Pure and Applied Mathematics*, 57(11):1413–1457.

Bradley Efron, Trevor Hastie, Iain Johnstone, and Robert Tibshirani. 2004. Least angle regression. *The Annals of Statistics*, 32(2):407–451.

Jerome H. Friedman, Trevor Hastie, and Rob Tibshirani. 2010. Regularization paths for generalized linear models via coordinate descent. *Journal of Statistical Software*, 33(1):1–22.

Arthur E. Hoerl and Robert W. Kennard. 1970. Ridge regression: Biased estimation for nonorthogonal problems. *Technometrics*, 12(1):55–67.

Seung-Jean Kim, Kwangmoo Koh, Michael Lustig, Stephen Boyd, and Dimitry Gorinevsky. 2007. An interior-point method for large-scale ℓ_1-regularized least squares. *IEEE Journal on Selected Topics in Signal Processing*, 1(4):606–617.

Eugene Ndiaye, Olivier Fercoq, Alexandre Gramfort, and Joseph Salmon. 2017. Gap Safe screening rules for sparsity enforcing penalties. *Journal of Machine Learning Research*, 18(128):1–33.

Brendan O'Donoghue and Emmanuel Candès. 2015. Adaptive restart for accelerated gradient schemes. *Foundations of Computational Mathematics*, 15:715–732.

Neal Parikh and Stephen Boyd. 2014. Proximal algorithms. *Foundations and Trends in Optimization*, 1(3).

scikit-learn developers. Diabetes dataset and Lasso. Documentation, release 1.9.1; accessed 5 October 2026.

Robert Tibshirani. 1996. Regression shrinkage and selection via the lasso. *Journal of the Royal Statistical Society: Series B (Methodological)*, 58(1):267–288.

Ryan J. Tibshirani. 2013. The lasso problem and uniqueness. *Electronic Journal of Statistics*, 7:1456–1490.

Paul Tseng. 2001. Convergence of a block coordinate descent method for nondifferentiable minimization. *Journal of Optimization Theory and Applications*, 109(3):475–494.

Stephen J. Wright. 2015. Coordinate descent algorithms. *Mathematical Programming*, 151(1):3–34.

Tong Tong Wu and Kenneth Lange. 2008. Coordinate descent algorithms for lasso penalized regression. *The Annals of Applied Statistics*, 2(1):224–244.

Peng Zhao and Bin Yu. 2006. On model selection consistency of Lasso. *Journal of Machine Learning Research*, 7(90):2541–2563.

Hui Zou. 2006. The adaptive lasso and its oracle properties. *Journal of the American Statistical Association*, 101(476):1418–1429.

Hui Zou and Trevor Hastie. 2005. Regularization and variable selection via the elastic net. *Journal of the Royal Statistical Society: Series B (Statistical Methodology)*, 67(2):301–320.